



Especificación Funcional de Pruebas Automatizadas en Karate

1. Documentación de Prerrequisitos

Para garantizar la correcta ejecución de este proyecto de automatización, el sistema local debe contar con los siguientes elementos instalados y configurados:

- **Java Development Kit (JDK):** Se requiere la versión **17 (LTS)**. Es fundamental que la variable de entorno `JAVA_HOME` apunte correctamente a la ruta de instalación de este JDK.
- **Gestor de Proyectos (Gradle):** El proyecto utiliza **Gradle**. Aunque se recomienda tener la versión **8.12.1** instalada globalmente, el proyecto incluye el `gradlew` (Gradle Wrapper), lo que permite ejecutar las tareas sin necesidad de una instalación previa de Gradle en el sistema.
- **Acceso a Red/API:** Dado que las pruebas validan servicios de autenticación, ubicación y pagos, se requiere conexión a los endpoints definidos en el archivo `karate-config.js`.

2. Especificación de Dependencias

A continuación se detallan las versiones exactas de las herramientas y frameworks que componen el stack tecnológico del proyecto:

Herramienta / Framework	Versión Especificada
Java	17.0.16" 2025-07-15 LTS
Karate Framework	2.2.2
Gradle	8.12.1
Protocolos de Pruebas	HTTP/S (REST API)

Nota: Las dependencias de librerías adicionales para la ejecución de BDD y reportes HTML están gestionadas automáticamente a través del archivo build.gradle.

3. Configuración del Entorno (Environment Setup)

Siga estos pasos para preparar el entorno antes de iniciar la ejecución de los casos de prueba:

1. **Clonación y Ubicación:** Descargue el proyecto y asegúrese de estar posicionado en la raíz, donde se encuentran los archivos build.gradle y gradlew.
2. **Configuración de Variables Globales:**
 - Verifique el archivo src/test/java/karate-config.js.
 - Asegúrese de que las variables de entorno (como baseUrl, tokens o credenciales de usuario) estén correctamente configuradas para el ambiente de pruebas deseado.
3. **Preparación de Datos:** El sistema utiliza archivos de datos externos (por ejemplo, userData y groupData) para alimentar los escenarios de **Gestión de Grupos de Ahorro y Actividad de Usuario**. Verifique que estos archivos JSON existan en las rutas correspondientes dentro de src/test/java.
4. **Verificación de Permisos:** En sistemas basados en Unix/Mac, asegúrese de que el wrapper de Gradle tenga permisos de ejecución con el comando: `chmod +x gradlew`.

1. Gestión de Autenticación y Acceso (auth.feature)

Objetivo: Garantizar la seguridad del ecosistema mediante el control estricto de identidades y sesiones de usuario.

- **Acceso Seguro:** Validación de ingreso mediante credenciales autorizadas y generación de tokens de sesión.
- **Persistencia de Sesión:** Verificación de la vigencia y estabilidad del token para una experiencia de usuario continua.
- **Control de Intrusiones:** Bloqueo de acceso a recursos privados ante intentos sin autorización o con credenciales erróneas.

- **Integridad de Registro:** Prevención de duplicidad de cuentas para mantener la base de datos de usuarios limpia.

2. Gestión de Información Demográfica (**demographics.feature**)

Objetivo: Proveer catálogos estandarizados para la caracterización precisa de los usuarios.

- **Estandarización de Perfiles:** Consulta de catálogos de género, estado civil, niveles educativos y etnias.
- **Inclusión y Calidad:** Verificación de que los catálogos de etnias y estratos sean completos y no contengan registros vacíos.
- **Robustez del Servicio:** Validación de manejo de errores ante rutas inexistentes o tipos de datos inválidos en los formularios.

3. Segmentación Geográfica (**location.feature**)

Objetivo: Estructurar la ubicación territorial de los usuarios para la operación logística y social.

- **Jerarquía Territorial:** Consulta multinivel de Países, Estados/Departamentos, Ciudades y Barrios.
- **Precisión de Datos:** Validación de la relación lógica entre zonas (ej. que las ciudades correspondan efectivamente al estado consultado).
- **Resiliencia Geográfica:** Control de respuestas ante parámetros no relacionados o fallos técnicos en la consulta de rutas.

4. Gestión de Pagos e Integración Financiera (**payment.feature**)

Objetivo: Asegurar la transparencia y exactitud en el flujo de dinero y cumplimiento de obligaciones.

- **Prevención de Cobros Indebidos:** El sistema impide la generación de firmas de pago si el usuario no presenta cuotas pendientes.
- **Validación de Transacciones:** Control de errores ante peticiones de pago con información insuficiente o inconsistente.
- **Protección del Usuario:** Mecanismos de seguridad contra pagos dobles o innecesarios.

5. Gestión de Grupos de Ahorro (saving_group.feature)

Objetivo: Administrar el ciclo de vida de las comunidades de ahorro (Círculos).

- **Ciclo de Vida del Grupo:** Pruebas integrales de creación, consulta de detalles y modificación de grupos de ahorro.
- **Reglas de Negocio:** Validación de quórum mínimo para activación de grupos y prevención de miembros duplicados.
- **Planificación Financiera:** Simulación de planes de pago y validación de coherencia en fechas de retorno.
- **Seguridad Estructural:** Verificación de accesos protegidos a las rutas de administración de grupos.

6. Actividad y Roles de Usuario (user.feature)

Objetivo: Controlar la visibilidad de la información y métricas según el perfil del usuario.

- **Seguridad Basada en Roles:** Validación de que solo usuarios autenticados accedan a sus grupos según su rol (Admin, Sub-Admin, Miembro).
- **Seguimiento Financiero:** Consulta de calendarios de pago mensuales y métricas de desempeño para el control de cumplimiento.
- **Privacidad de Datos:** Restricción de acceso a información de actividad para peticiones no autorizadas.



Guía de Conceptos de Pruebas Automatizadas (Karate/Gherkin)

Estos términos representan el "paso a paso" lógico de cada prueba y aseguran que el sistema se comporte como el negocio espera.



Background (Antecedentes)

Es la configuración previa que se repite en todas las pruebas de un mismo archivo.

- **En lenguaje de negocio:** "Antes de empezar cualquier acción, asegúrate de estar en el lugar correcto (URL) y tener los permisos necesarios".



Given (Dado que...)

Define el contexto inicial o la situación de partida. Establece los requisitos previos.

- **En lenguaje de negocio:** "Dado que el usuario tiene un correo y una contraseña válidos..." o "Dado que estamos consultando el catálogo de países...".

● **And (Y además...)**

Se utiliza para añadir condiciones adicionales de forma fluida, ya sea en el contexto o en las acciones.

- **En lenguaje de negocio:** "...y además el usuario tiene el rol de Administrador".

● **When (Cuando...)**

Describe la acción clave que el usuario o el sistema realiza. Es el evento que desencadena la respuesta.

- **En lenguaje de negocio:** "Cuando el usuario hace clic en 'Iniciar Sesión'" o "Cuando el sistema solicita el listado de ciudades".

● **Then (Entonces...)**

Es el resultado esperado. Aquí es donde se verifica que el sistema hizo lo correcto.

- **En lenguaje de negocio:** "Entonces el sistema debe permitir el ingreso" o "Entonces el catálogo debe mostrar 6 estratos socioeconómicos".

○ **Match (Validación/Coincidencia)**

Es una instrucción específica de Karate para comparar que la respuesta sea **exactamente** lo que el negocio definió.

- **En lenguaje de negocio:** "Verifica que el mensaje de error sea 'No autorizado' y que el formato de la respuesta sea el acordado".

🔍 **Ejemplo:**

Escenario: Inicio de sesión exitoso.

- **Given (Dado que):** Tengo mis credenciales.
- **When (Cuando):** Las envío al sistema.
- **Then (Entonces):** El sistema me responde con un código 201 (Éxito).
- **Match (Verificar):** Y compruebo que me entregó un token de seguridad válido.